Generative Methods

Images

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 21, 2026

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

Outline

1. Generative vs. Discriminative
2. Generative Models for Images
3. Variational Autoencoder (VAE)
4. Generative Adversarial Networks (GANs)
5. Denoising diffusion (DDPM)
6. Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Generative vs. Discriminative

Generative Methods

Outline

2 Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Motivation

Why generative models?

Discriminative vs. generative

- **Discriminative:** learn $p(\boldsymbol{y} \mid \boldsymbol{x})$ or a decision boundary (e.g. classifiers).
- **Generative:** learn a distribution over data, typically $p(\boldsymbol{x})$ or $p(\boldsymbol{x}, \boldsymbol{y})$.

Why do we care about $p(x)$?

- **Sampling:** draw new realistic images, sentences, or tabular records.
- **Data augmentation:** synthesize extra examples for rare classes.
- **Imputation & denoising:** fill in missing values, remove noise.
- **Compression / representation learning:** learn useful latent codes.

Generative Methods
Outline
3 Generative vs. Discriminative
Generative Models for Images
Variational Autoencoder (VAE)
Generative Adversarial Networks (GANs)
Denoising diffusion (DDPM)
Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Probability view

Joint, conditional, and marginal

- Suppose $(\boldsymbol{x}, y)$ with input $\boldsymbol{x}$ and label y.
- **Joint:** $p(\boldsymbol{x}, y)$, **marginal:** $p(\boldsymbol{x}) = \sum_y p(\boldsymbol{x}, y)$.
- **Conditionals:**

$$p(y \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x}, y)}{p(\boldsymbol{x})}, \qquad p(\boldsymbol{x} \mid y) = \frac{p(\boldsymbol{x}, y)}{p(y)}.$$

- Generative models can be used for classification via

$$p(y \mid \boldsymbol{x}) \propto p(\boldsymbol{x} \mid y)\, p(y),$$

but in modern deep learning we often focus on generation and representation.

Generative Methods
Outline
4 Generative vs. Discriminative
Generative Models for Images
Variational Autoencoder (VAE)
Generative Adversarial Networks (GANs)
Denoising diffusion (DDPM)
Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Generative Models for Images

Generative Methods

Outline

Generative vs. Discriminative

5 Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Images as data

What do we want to generate?

- Image $\boldsymbol{x}$ can be seen as a vector in $\mathbb{R}^{H \times W \times C}$.
- We want to learn $p(\boldsymbol{x})$ such that samples $\tilde{\boldsymbol{x}} \sim p$:
  - look realistic to humans,
  - cover the main modes of the data distribution (no trivial memorization),
  - can be controlled (e.g. conditional on class label).
- Typical datasets: MNIST, Fashion-MNIST, CIFAR-10, CelebA, ImageNet (hard).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Families of image generators

High-level landscape

- **Autoencoders (AE):** encoder $\boldsymbol{x} \mapsto \boldsymbol{z}$, decoder $\boldsymbol{z} \mapsto \hat{\boldsymbol{x}}$ (typically a **deterministic** bottleneck).
- **Variational Autoencoders (VAE):** probabilistic AE with latent prior $p(\boldsymbol{z})$ and ELBO training.
- **Generative Adversarial Networks (GAN):** generator $G(\boldsymbol{z})$ vs. discriminator $D(\boldsymbol{x})$ in a minimax game.
- **Diffusion / score-based models:** gradually add noise then learn to reverse the process.
- **Autoregressive models:** factorize $p(\boldsymbol{x})$ pixel-by-pixel (PixelCNN, PixelRNN).

ltsach@hcmut.edu.vn
Thanh-Sach LE

MNIST demos: repository layout

samples/ (course code)

Paths (relative to the Chapter 9 slides folder)

- `samples/vae/` — Variational Autoencoder on flattened MNIST (`train_mnist_vae.py`, `config.yaml`).
- `samples/gan/` — GAN with **MLP** or **DCGAN**-style conv (`train_mnist_gan.py`; `config.yaml` vs. `config_dcgan.yaml`).
- `samples/diffusion/` — DDPM with a small U-Net (`train_mnist_diffusion.py`, `config.yaml`).

Common pattern

Each folder: `requirements.txt`, YAML config, script, and a `results/` directory with figures + `results.txt` (metrics and paths).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Data protocol

Train / validation / test (MNIST)

- Official MNIST **training** set (60,000 images) is split into:
  - **train** fraction = `train_ratio` (e.g. 0.8–0.9) for optimization,
  - **validation** = remainder, for model selection (best checkpoint).
- Official MNIST **test** set (10,000) is held out for **final** evaluation only (same spirit as the RNN MNIST demo).
- All three demos normalize pixels consistently where needed (e.g. VAE: $[0, 1]$; GAN / diffusion: often $[-1, 1]$ for Tanh or Gaussian noise).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Outputs you should inspect

What to show in class

Demo	Typical artifacts under `results/`
VAE	`samples.png` (prior samples), `reconstructions.png` (input vs. reconstruction), `loss_curve.png`, `results.txt`
GAN	`samples.png` (generator grid), `loss_curve.png` (D/G losses), checkpoints `best_G.pt` / `best_D.pt`
Diffusion	`samples.png` (reverse-process samples), `loss_curve.png` (noise MSE), `checkpoints/best_ddpm.pt`

Side-by-side: VAE (often softer) vs. GAN (sharper but unstable) vs. diffusion (sharpness vs. compute trade-off).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Variational Autoencoder (VAE)

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

11 Variational Autoencoder (VAE)

Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

What we want (big picture)

- Data $\boldsymbol{x}$ is high-dimensional, but $p_{\text{data}}(\boldsymbol{x})$ lies on a **thin manifold**.
- Goals:
  - **Generate** new samples $\boldsymbol{x}$
  - Learn a **compact latent** representation $\boldsymbol{z}$
- Key idea:
  - Introduce latent $\boldsymbol{z}$ with simple prior $p(\boldsymbol{z})$ (e.g. $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$)
  - Learn how data are generated via $\boldsymbol{z} \to \boldsymbol{x}$

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

12 Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Autoencoder (AE)

- Deterministic mapping:

$$\boldsymbol{z} = f(\boldsymbol{x}), \quad \hat{\boldsymbol{x}} = g(\boldsymbol{z})$$

- Learns compression / reconstruction
- **No explicit generative model**
- Cannot sample $\boldsymbol{z}$ to generate new data

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

13 Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Variational Autoencoder (VAE)

- Encoder outputs a **distribution**:

$$q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$$

- Sample $\boldsymbol{z} \sim q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$
- Decoder:

$$p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$$

- **Generative path:**

$$\boldsymbol{z} \sim p(\boldsymbol{z}) \rightarrow \boldsymbol{x} \sim p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$$

ltsach@hcmut.edu.vn

VAE

Key difference

AE

- Deterministic mapping $\boldsymbol{x} \to \boldsymbol{z}$
- No explicit generative story

VAE

- Probabilistic encoder $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$
- Can sample $\boldsymbol{z} \sim p(\boldsymbol{z})$ to generate

ltsach@hcmut.edu.vn

VAE

Generative model (conceptual view)

Ingredients:

- **Prior** $p(\boldsymbol{z})$ (often $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$) — where unconditional samples come from.
- **Decoder** $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$ — how $\boldsymbol{x}$ is generated from $\boldsymbol{z}$.
- **Encoder (training only)** $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ — approximate posterior; we do not need it for pure generation after training.

Pipeline (training): $\boldsymbol{x} \xrightarrow{\text{encoder}} q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \xrightarrow{\text{sample}} \boldsymbol{z} \xrightarrow{\text{decoder}} \hat{\boldsymbol{x}}$.

- $\boldsymbol{z}$ is **random**, not a single deterministic code — we fit and regularize the whole family $q_\phi(\cdot \mid \boldsymbol{x})$.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

16 Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

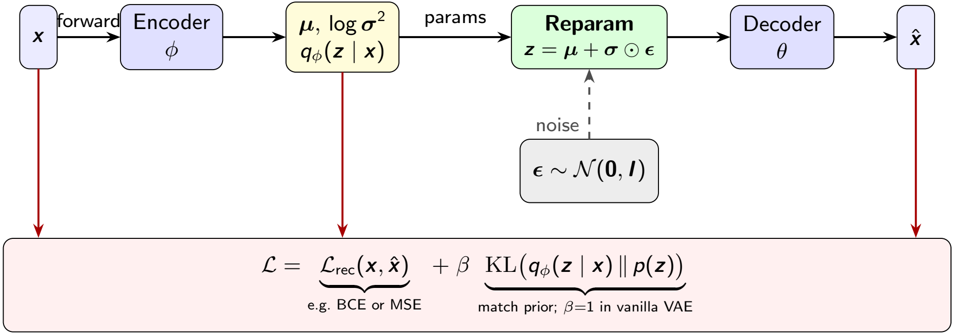
Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Training graph (computational view)



- **Trainable:** ϕ and θ appear on *differentiable* paths; sampling z is non-differentiable, but **reparameterization** routes gradients w.r.t. ϕ through (μ, σ).
- **Objective:** minimize $\mathcal{L}$; β controls reconstruction vs. regularization (standard ELBO uses $\beta = 1$).

ltsach@hcmut.edu.vn

VAE

Using the model: reconstruct, generate, and encode

- **Reconstruct:** $\boldsymbol{x} \to$ encoder $\to \boldsymbol{z}$ (often $\boldsymbol{z} = \boldsymbol{\mu}$) $\to$ decoder $\to \hat{\boldsymbol{x}}$; or sample $\boldsymbol{z} \sim q_\phi$ for stochastic reconstructions.
- **Generate:** $\boldsymbol{z} \sim p(\boldsymbol{z})$, then $\hat{\boldsymbol{x}}$ from decoder only — **no encoder**.
- **Encode / latents:** $\boldsymbol{x} \mapsto \boldsymbol{z}$ via encoder for visualization, features, or downstream tasks.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Two terms in words (before heavy math)

- **Reconstruction:** make $\hat{\boldsymbol{x}}$ close to $\boldsymbol{x}$ (in code: often BCE with sigmoid pixels, or MSE under a Gaussian decoder).
- **Regularization:** keep $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ close to prior $p(\boldsymbol{z})$ so latents stay in a region where sampling works — implemented as **KL divergence**.
- **Trade-off:** fidelity vs. structured latent space (often weighted by β in β-VAE).
- **Reparameterization:** sample $\boldsymbol{z}$ so gradients reach encoder parameters (derived in the mathematical view).

ltsach@hcmut.edu.vn

VAE on MNIST

samples/vae/ — tie ideas to code

What the script implements

- Flatten 28×28; MLP encoder $\rightarrow (\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2)$; $\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \epsilon$; MLP decoder $\rightarrow \hat{\boldsymbol{x}}$ with **sigmoid**; loss = reconstruction (BCE-style) + KL (Gauss–Gauss).
- `config.yaml`: `latent_dim`, `hidden_dims`, `beta`, `epochs`, `train_ratio`; train/val on 60k, test on 10k.

Inspect

`reconstructions.png`, `samples.png`, `loss_curve.png`, `results.txt`.

ltsach@hcmut.edu.vn

Why VAE samples often look "soft"

Beyond "train longer"

- Factorized likelihood / ELBO can favor **averages** across plausible shapes $\Rightarrow$ **blurry** edges.
- More epochs help only partly; perceptual sharpness is not the main selling point vs. GANs / diffusion.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Conceptual view (intuition)

21 Computational view (training and usage)

Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Step 0: what we cannot compute directly

Generative model: prior $p(\boldsymbol{z})$, decoder $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$.

1. **Marginal density / evidence:** $p_\theta(\boldsymbol{x}) = \int p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})\, \mathrm{d}\boldsymbol{z}$. For fixed $\boldsymbol{x}$, as a function of θ, the same expression is the **marginal likelihood** $L(\theta)$ (often loosely called the **likelihood** in MLE). High-dimensional integral $\Rightarrow$ no closed form for flexible $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$.
2. **True posterior:** $p_\theta(\boldsymbol{z} \mid \boldsymbol{x}) = \dfrac{p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})}{p_\theta(\boldsymbol{x})}$. Unknown $p_\theta(\boldsymbol{x})$ in the denominator $\Rightarrow$ posterior also intractable.
3. **Variational idea:** introduce an *approximate* posterior $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ (encoder) and reason with Kullback–Leibler divergences instead.

ltsach@hcmut.edu.vn

VAE

Step 1: importance identity (true equality)

Fix data $\boldsymbol{x}$. Choose $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ that covers all relevant latent regions (so we do not divide by zero where $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})p(\boldsymbol{z})$ is nonzero).

$$
\begin{aligned}
p_\theta(\boldsymbol{x}) &= \int p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})\, \mathrm{d}\boldsymbol{z} && \text{marginal: integrate out } \boldsymbol{z} \text{ under } p(\boldsymbol{z}) \\
&= \int \frac{p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})}{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})}\, q_\phi(\boldsymbol{z} \mid \boldsymbol{x})\, \mathrm{d}\boldsymbol{z} && \text{multiply \& divide by } q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \\
&= \mathbb{E}_{\boldsymbol{z} \sim q_\phi(\cdot \mid \boldsymbol{x})}\left[\frac{p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})}{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})}\right] && \int (\cdot)\, q_\phi\, \mathrm{d}\boldsymbol{z} = \mathbb{E}_{q_\phi}[\cdot]
\end{aligned}
$$

Take log:

$$
\log p_\theta(\boldsymbol{x}) = \log \mathbb{E}_{q_\phi}\left[\frac{p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})}{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})}\right].
$$

Still hard: log remains *outside* the expectation.

VAE

Step 2: Jensen $\Rightarrow$ a tractable lower bound

For any positive random variable Y, log is **concave**, so (Jensen)

$$\log \mathbb{E}[Y] \;\geq\; \mathbb{E}[\log Y].$$

Apply with $Y(\boldsymbol{z}) = \dfrac{p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\, p(\boldsymbol{z})}{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})}$:

$$\log p_\theta(\boldsymbol{x}) \;\geq\; \mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) + \log p(\boldsymbol{z}) - \log q_\phi(\boldsymbol{z} \mid \boldsymbol{x})] =: \mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{x}).$$

This is the **ELBO** (evidence lower bound). Group terms:

$$\mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{x}) = \mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})] - \mathbb{E}_{q_\phi}\left[\log \frac{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})}{p(\boldsymbol{z})}\right].$$

By definition $\mathrm{KL}(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})) = \mathbb{E}_{q_\phi}\left[\log \frac{q_\phi(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right]$, so

$\mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{x}) = \mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})] - \mathrm{KL}(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}))$.

ltsach@hcmut.edu.vn

VAE

Step 3: read ELBO term-by-term

$$\mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{x}) = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})]}_{\text{(A) fit data after decoding}} - \underbrace{\mathrm{KL}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\big)}_{\text{(B) keep latent posterior close to prior}} .$$

1. **(A) Reconstruction/likelihood term:** encourages decoder parameters θ to make $\boldsymbol{x}$ probable from sampled $\boldsymbol{z}$.
2. **(B) KL term:** regularizes encoder output so latent codes stay in regions where sampling from $p(\boldsymbol{z})$ is meaningful.
3. **Optimization target:** maximize ELBO (equivalently minimize $-\mathcal{L}_{\mathrm{ELBO}}$) over both θ and ϕ.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Conceptual view (intuition)

Computational view (training and usage)

25 Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Step 4: why ELBO is a lower bound

Why this KL appears: we already introduced an approximate posterior $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ for the true posterior $p_\theta(\boldsymbol{z} \mid \boldsymbol{x})$. So a natural "gap" measure is $\mathrm{KL}(q_\phi \| p_\theta(\cdot \mid \boldsymbol{x}))$. Using Bayes: $p_\theta(\boldsymbol{z} \mid \boldsymbol{x}) = \dfrac{p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) p(\boldsymbol{z})}{p_\theta(\boldsymbol{x})}$. **Now derive the exact gap (fix one data point $\boldsymbol{x}$):**

$$\begin{aligned}
\mathrm{KL}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \| p_\theta(\boldsymbol{z} \mid \boldsymbol{x})\big) &= \mathbb{E}_{q_\phi}\left[\log \frac{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})}{p_\theta(\boldsymbol{z} \mid \boldsymbol{x})}\right] \\
&= \mathbb{E}_{q_\phi}[\log q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) - \log p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) - \log p(\boldsymbol{z}) + \log p_\theta(\boldsymbol{x})] \\
&= \log p_\theta(\boldsymbol{x}) - \mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})] + \mathbb{E}_{q_\phi}[\log q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) - \log p(\boldsymbol{z})] \\
&= \log p_\theta(\boldsymbol{x}) - \mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{x}).
\end{aligned}$$

Rearrange:

$$\log p_\theta(\boldsymbol{x}) = \mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{x}) + \mathrm{KL}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \| p_\theta(\boldsymbol{z} \mid \boldsymbol{x})\big).$$

Since $\mathrm{KL} \geq 0$, ELBO is a lower bound; equality when $q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) = p_\theta(\boldsymbol{z} \mid \boldsymbol{x})$.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Conceptual view (intuition)

Computational view (training and usage)

26 Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

VAE

Key insight (bridge to training)

- From Step 4: $\log p_\theta(\boldsymbol{x}) = \mathcal{L}_{\text{ELBO}}(\boldsymbol{x}) + \text{KL}(q_\phi \| p_\theta(\cdot \mid \boldsymbol{x}))$ — *marginal log-likelihood* equals ELBO plus **posterior mismatch**.
- So maximizing ELBO pushes up a tractable lower bound while encouraging q_ϕ to track $p_\theta(\boldsymbol{z} \mid \boldsymbol{x})$.
- Joint training fits **both** the generative model p_θ (decoder) and the inference model q_ϕ (encoder).

ltsach@hcmut.edu.vn

VAE

Step 5: from ELBO to trainable loss

1. **Sampling trick (reparameterization):**

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \quad \boldsymbol{z} = \boldsymbol{\mu}_\phi(\boldsymbol{x}) + \boldsymbol{\sigma}_\phi(\boldsymbol{x}) \odot \boldsymbol{\epsilon}.$$

This writes random $\boldsymbol{z}$ as a differentiable transform of (μ, σ) and noise.

2. **Mini-batch objective (negative ELBO, one MC sample):**

$$\mathcal{J}(\theta, \phi; \boldsymbol{x}) \approx -\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) + \mathrm{KL}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\big).$$

In code, $-\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$ is implemented as a reconstruction loss (typically BCE for Bernoulli decoder, or MSE for Gaussian decoder).

3. **β-VAE variant:**

$$\mathcal{J}_\beta \approx -\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) + \beta\, \mathrm{KL}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\big), \quad \beta > 0.$$

Larger β gives stronger latent regularization.

Generative Methods
Outline
Generative vs. Discriminative
Generative Models for Images
Variational Autoencoder (VAE)
Conceptual view (intuition)
Computational view (training and usage)
28 Mathematical view (ELBO and training)
Generative Adversarial Networks (GANs)
Denoising diffusion (DDPM)
Summary
56
ltsach@hcmut.edu.vn

VAE

ELBO roadmap (step-by-step recap)

1. Start from intractable $p_\theta(\boldsymbol{x})$ and $p_\theta(\boldsymbol{z} \mid \boldsymbol{x})$.
2. Introduce $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$ and rewrite $p_\theta(\boldsymbol{x})$ as an expectation under q_ϕ.
3. Apply Jensen to obtain ELBO.
4. Split ELBO into likelihood fit $-$ KL regularization.
5. Use reparameterization to estimate gradients and optimize with SGD/Adam.

One-line memory: *fit data in observation space, regularize structure in latent space.*

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Conceptual view (intuition)

Computational view (training and usage)

29 Mathematical view (ELBO and training)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

Generative Adversarial Networks (GANs)

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

30 Generative Adversarial Networks (GANs)

Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (objective and training)

Denoising diffusion (DDPM)

Summary

56

ltsach@hcmut.edu.vn

GANs

What we want (big picture)

- Learn a direct sampling map from noise to data:

$$\boldsymbol{z} \sim p(\boldsymbol{z}), \qquad \tilde{\boldsymbol{x}} = G_\theta(\boldsymbol{z}).$$

- No explicit encoder and no tractable likelihood $p_\theta(\boldsymbol{x})$ as in VAE.
- Training signal comes from a learned critic/classifier D_ϕ that distinguishes real vs. fake.

ltsach@hcmut.edu.vn

GANs

Two networks, one game

1. **Generator** G_θ**:** synthesis path only, $\boldsymbol{z} \mapsto \tilde{\boldsymbol{x}}$.
2. **Discriminator** D_ϕ**:** outputs a realness score for an input image.
3. **Competition:** D tries to separate real/fake; G tries to fool D.

Intuition: as D improves, it provides better gradients; as G improves, fake samples move toward p_{data}.

Outline
Generative vs. Discriminative
Generative Models for Images
Variational Autoencoder (VAE)
Generative Adversarial Networks (GANs)
32 Conceptual view (intuition)
Computational view (training and usage)
Mathematical view (objective and training)
Denoising diffusion (DDPM)
Summary

ltsach@hcmut.edu.vn

GANs

GAN training diagram

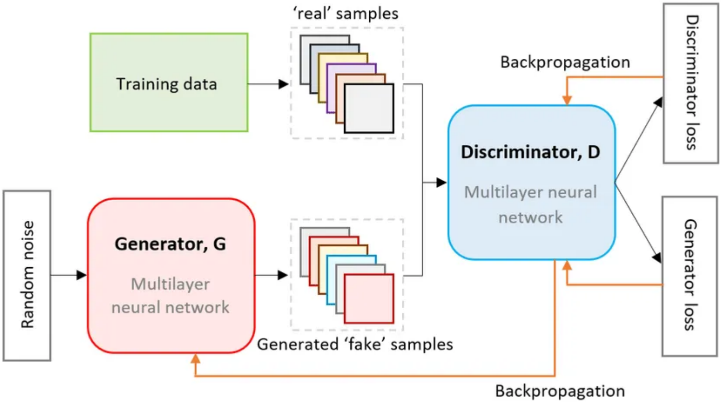


Figure 4.1: GAN training flow: noise → generator G; training data vs. generated fakes → discriminator D; separate losses and backpropagation to D and G. Source: Presidio, Exploring the Power of Generative Adversarial Networks (GANs) with Azure.

ltsach@hcmut.edu.vn

GANs

Using the model after training

- **Generation only:** sample $\boldsymbol{z} \sim p(\boldsymbol{z})$, output $\tilde{\boldsymbol{x}} = G_\theta(\boldsymbol{z})$.
- **No encoder in vanilla GAN:** unlike VAE, there is no direct $\boldsymbol{x} \mapsto \boldsymbol{z}$ path.
- **No discriminator needed at inference:** D is mainly a training-time teacher.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (objective and training)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

GANs

What code actually optimizes

1. **BCE with logits** for D (stable); often alternate k steps on D vs. one on G.
2. **Non-saturating** G**:** maximize $\mathbb{E}[\log D(G(\boldsymbol{z}))]$ instead of $\mathbb{E}[\log(1 - D(G(\boldsymbol{z})))]$ to avoid vanishing gradients.
3. **Label smoothing** on real targets (e.g. 0.9 instead of 1) to reduce overconfident D.
4. **Balance:** if D is too strong $\Rightarrow$ weak gradients for G; tune `lr_d`, `lr_g`, capacities, update ratio.

ltsach@hcmut.edu.vn

GAN on MNIST (course code)

samples/gan/ — tie configs to outputs

Architectures

- `config.yaml` (`arch: mlp`): G, D MLP on flattened 784-D, Tanh pixels in $[-1, 1]$ — fast baseline; may need more tuning.
- `config_dcgan.yaml` (`arch: dcgan`): conv-transpose G + conv D on $1 \times 28 \times 28$; use e.g. `results_dcgan/` to compare side-by-side.

What to inspect

- Selection: val loss on D (real/fake BCE combined) — simple scalar, **not** FID.
- Artifacts: `best_G.pt`, `best_D.pt`; grid `samples.png`; curves `loss_curve.png`.

ltsach@hcmut.edu.vn

GANs

Step 0: minimax objective (original)

$$\min_{\theta} \max_{\phi} \; \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}}[\log D_\phi(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}[\log(1 - D_\phi(G_\theta(\boldsymbol{z})))].$$

- For fixed G, the best D approximates a density-ratio classifier between real and generated data.
- At equilibrium, G pushes generated distribution toward p_{data} (in the Jensen–Shannon sense for the vanilla objective).

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Conceptual view (intuition)

Computational view (training and usage)

37 Mathematical view (objective and training)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

GANs

Step 1: practical generator objective

- Original G term $\mathbb{E}[\log(1 - D(G(\boldsymbol{z})))]$ can saturate when D is very confident.
- Practical replacement (non-saturating):

$$\max_{\theta} \mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}[\log D_{\phi}(G_{\theta}(\boldsymbol{z}))].$$

- Same fixed point intuition, usually better gradients early in training.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Conceptual view (intuition)

Computational view (training and usage)

38 Mathematical view (objective and training)

Denoising diffusion (DDPM)

Summary

ltsach@hcmut.edu.vn

GANs

Roadmap + trade-offs

1. Sample $\boldsymbol{z}$; synthesize $\tilde{\boldsymbol{x}} = G(\boldsymbol{z})$; train D on real vs. fake.
2. Optimize minimax (or practical BCE / non-saturating variants).
3. At inference: **only** G — one forward pass per sample.

- **Pros:** often sharp samples; cheap generation after training.
- **Cons:** unstable training, mode collapse; no tractable $p_\theta(\boldsymbol{x})$; many variants (WGAN, StyleGAN,...).

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Conceptual view (intuition)

Computational view (training and usage)

39 Mathematical view (objective and training)

Denoising diffusion (DDPM)

Summary

56

ltsach@hcmut.edu.vn

Denoising diffusion (DDPM)

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

40 Denoising diffusion (DDPM)

Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (optional deeper details)

Summary

ltsach@hcmut.edu.vn

DDPM

What we want (big picture)

- Learn to generate data by **reversing noise**.
- Instead of one-shot generation (GAN), DDPM uses a **multi-step refinement** process.
- Core intuition:
  1. Corrupt real images gradually with Gaussian noise until nearly pure noise.
  2. Train a neural network to predict and remove noise at each step.
  3. At inference, start from random noise and denoise step-by-step to obtain a sample.

ltsach@hcmut.edu.vn

DDPM

Why diffusion is attractive

- **Stable training objective:** supervised noise prediction (MSE-style), no adversarial minimax game.
- **High sample quality:** often competitive visual fidelity.
- **Trade-off:** generation is slower because sampling needs many reverse steps (T evaluations).
- Practical lens for students:
  - GAN = fast generation, harder training.
  - Diffusion = easier training, slower generation.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

42 Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (optional deeper details)

Summary

ltsach@hcmut.edu.vn

DDPM for beginners

Minimum you must remember

1. **Forward noising:** gradually add Gaussian noise to clean image $\boldsymbol{x}_0$ until almost pure noise $\boldsymbol{x}_T$.
2. **Train target:** given $(\boldsymbol{x}_t, t)$, the model predicts the injected noise ϵ.
3. **Loss:** simple regression loss (typically MSE/L2) between true noise and predicted noise.
4. **Generation:** start from $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, denoise step-by-step to get $\boldsymbol{x}_0$.
5. **Trade-off:** usually stable and high quality, but sampling is slower than one-shot generators.

One-line memory

DDPM = "add noise with a known rule" + "learn to remove it step-by-step."

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

43 Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (optional deeper details)

Summary

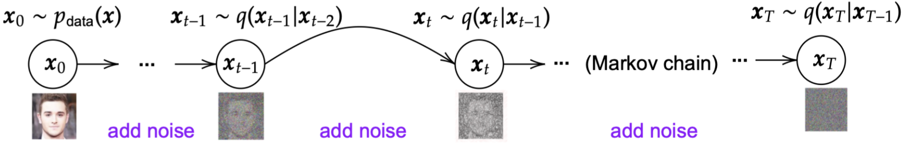
ltsach@hcmut.edu.vn

DDPM forward process

From $\mathbf{x}_0$ to $\mathbf{x}_t$ with key formulas

Forward

$\boldsymbol{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$ when: T is large.



β_t : *small variance*; $\alpha_t = 1 - \beta_t$

$\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \qquad t \in [1, T] \subset \mathbb{N}^+$

$$\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}\left(\sqrt{\overline{\alpha}_t}\, \boldsymbol{x}_0,\ (1 - \overline{\alpha}_t)\mathbf{I}\right)$$

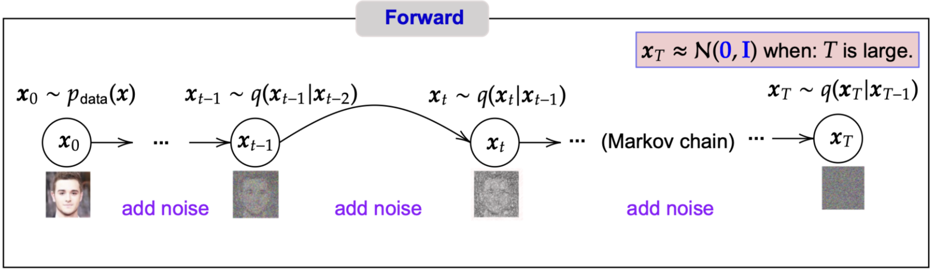
$$\boldsymbol{x}_t = \sqrt{\overline{\alpha}_t}\boldsymbol{x}_0 + \sqrt{(1 - \overline{\alpha}_t)}\boldsymbol{\epsilon}_t$$

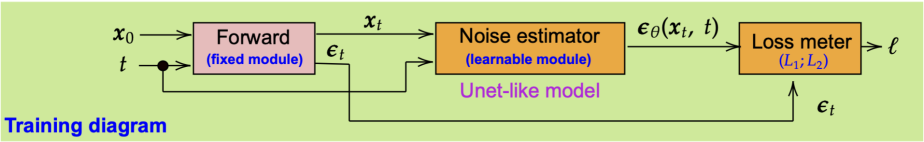
Can compute x_t directly from x_0

ltsach@hcmut.edu.vn

DDPM training view

Forward module + learnable noise estimator





Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

45 Conceptual view (intuition)

Computational view (training and usage)

Mathematical view (optional deeper details)

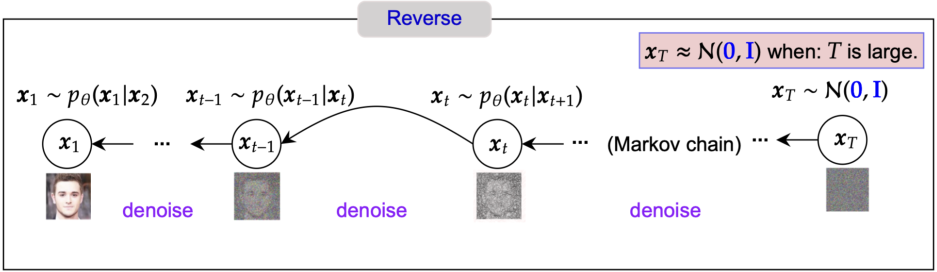
Summary

56

ltsach@hcmut.edu.vn

DDPM reverse process

Sampling by iterative denoising



Algorithm 2 Sampling

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = T, \ldots, 1$ **do**
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
  $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
end for
return $\mathbf{x}_0$

ltsach@hcmut.edu.vn

DDPM

Training/inference pipeline (no heavy math yet)

Training loop

1. Sample clean image $\boldsymbol{x}_0$ and random timestep $t \in \{1, \ldots, T\}$.
2. Add noise to obtain $\boldsymbol{x}_t$.
3. Predict noise $\hat{\epsilon} = \epsilon_\theta(\boldsymbol{x}_t, t)$.
4. Minimize prediction error against true ϵ.

Generation loop

Start from $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, then iteratively apply learned reverse updates $\boldsymbol{x}_T \to \boldsymbol{x}_{T-1} \to \cdots \to \boldsymbol{x}_0$.

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Conceptual view (intuition)

47 Computational view (training and usage)

Mathematical view (optional deeper details)

Summary

56

ltsach@hcmut.edu.vn

DDPM on MNIST (course code)

samples/diffusion/

Implementation choices

- Images scaled to $[-1, 1]$; linear β schedule from `beta_start` to `beta_end` over `timesteps` T.
- Small **U-Net** with residual blocks + **sinusoidal time embedding** + downsampling to 7×7 and upsampling with skip connections.
- Train/val/test: same MNIST split as other demos; best checkpoint by **val MSE** on noise prediction; test MSE reported in `results.txt`.
- **Cost:** sampling runs T network evaluations $\Rightarrow$ slower than one-shot $G(\boldsymbol{z})$ in GANs.

Classroom usage

If students are weak in math, focus on this computational recipe first (and skip detailed derivations in the next subsection).

Generative Methods

Outline

Generative vs. Discriminative

Generative Models for Images

Variational Autoencoder (VAE)

Generative Adversarial Networks (GANs)

Denoising diffusion (DDPM)

Conceptual view (intuition)

48 Computational view (training and usage)

Mathematical view (optional deeper details)

Summary 56

ltsach@hcmut.edu.vn

DDPM

Math roadmap (optional)

- Step 1: define forward Gaussian noising process.
- Step 2: derive closed-form $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$.
- Step 3: parameterize reverse process with $\epsilon_\theta(\boldsymbol{x}_t, t)$.
- Step 4: use simplified training loss $\mathcal{L}_{\text{simple}}$.

Teaching note

Students can skip proof details and remember only: "*sample t, add noise, predict noise*".

ltsach@hcmut.edu.vn

DDPM

Step 1–2: forward process and closed-form

- Fix a horizon T and a variance schedule $\beta_1, \ldots, \beta_T$ (small positive).
- Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.
- Forward process (Gaussian transitions):

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}\big(\sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1},\, \beta_t \boldsymbol{I}\big), \qquad q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \approx \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}).$$

- Closed-form at any t:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\, (1-\bar{\alpha}_t)\boldsymbol{I}\big).$$

ltsach@hcmut.edu.vn

DDPM

Step 3–4: reverse process and objective

- Learn a network $\epsilon_\theta(\boldsymbol{x}_t, t)$ to predict the **noise** ϵ in

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}).$$

- Training loss (simple DDPM):

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,\boldsymbol{x}_0,\epsilon}\left\|\epsilon - \epsilon_\theta(\boldsymbol{x}_t, t)\right\|_2^2.$$

- Sampling: start from $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, run T learned reverse steps (Markov Gaussian updates) to obtain $\boldsymbol{x}_0$.

ltsach@hcmut.edu.vn

DDPM advanced math

Core equations (optional)

1) Forward transition

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1},\, \beta_t \boldsymbol{I}\right), \quad \alpha_t = 1 - \beta_t,\ \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

2) Closed-form noising at any step

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\, (1-\bar{\alpha}_t)\boldsymbol{I}\right), \quad \boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},$$

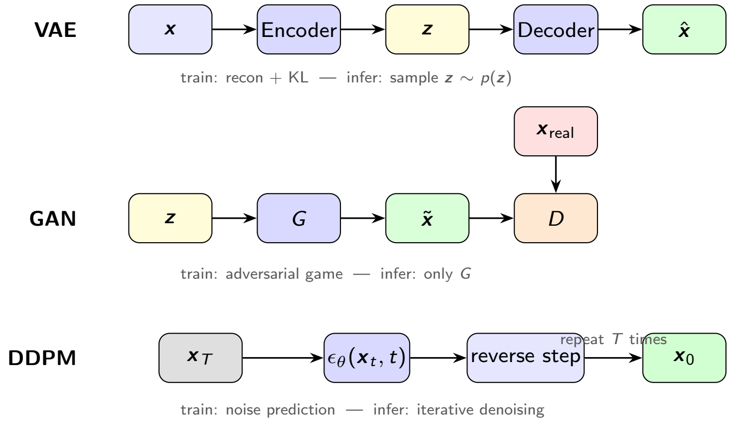
$$\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}).$$

3) Practical training loss

ltsach@hcmut.edu.vn

Pipeline comparison

VAE vs. GAN vs. DDPM (at a glance)



ltsach@hcmut.edu.vn

Diffusion vs. VAE vs. GAN

Qualitative comparison (MNIST demos)

	VAE	GAN	DDPM
Likelihood	ELBO / approximate	no explicit $p(\boldsymbol{x})$	sequence of Gaussian steps
Sharpness	often softer	often sharper (if stable)	can be sharp; depends on T, schedule, capacity
Training	relatively stable	can be unstable	stable but slow sampling

Outline
Generative vs. Discriminative
Generative Models for Images
Variational Autoencoder (VAE)
Generative Adversarial Networks (GANs)
Denoising diffusion (DDPM)
Conceptual view (intuition)
Computational view (training and usage)
54 Mathematical view (optional deeper details)
Summary
56

ltsach@hcmut.edu.vn

Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

Summary

Key takeaways

Concepts

- Generative models aim to learn $p(\boldsymbol{x})$ or $p(\boldsymbol{x}, \boldsymbol{y})$, enabling sampling and data synthesis.
- **Images:** VAE (ELBO, often blurry samples), GAN (adversarial game, sharp but fragile), DDPM (noise prediction + iterative sampling).
- **Course alignment (images):** runnable MNIST baselines in `samples/vae`, `samples/gan` (MLP + DCGAN configs), `samples/diffusion` with YAML configs and `results/` artifacts for lectures.
- **Text generation** (tokenizers, causal LM training, decoding, KV-cache, code samples): covered in `slides/10-modern-llms/ModernLLMs.tex` — see pointers at the end of the diffusion section in this deck.
- Conditional generation and data synthesis have many applications but also raise ethical and societal concerns.

ltsach@hcmut.edu.vn
Thanh-Sach LE